

Wk 2 Lab log

Wk 2 summary

This week involved myself reading up on more of the theory, building up the dataset object, and debugging some of the basic observables. Lab partner began writing code for cluster analysis.

28/01/2026 note:

Have begun switching from onenote labbook to a quarto document.

Lab log 29/01/2026

Dr Turci session; went over Jaynes model and mean field theory derivation
catchup w/lab partner; outlined code She summarised lit review:

- cluster definition:
 - useful definition; connected points w/same spin; (Domb and Stoll 1977)

code plan:

- dataset object
 - list of config objects over temperature
 - save as folder of config objects - give config objects useful names to reconstruct dataset object
- getClusters - returns list of clusters as some sort of data structure (maybe True/False bool array w/spin value?)
- cluster analysis methods:
 - cluster size
 - correlation length?
 - cluster mean position? may not be useful for debugging?
 - cluster spread (standard deviation) - measure of how spread out

cluster selection algorithm outline:

may need two methods; one “find cluster from point” method and one “find all clusters” method

findAllClusters():

- create bool array of “checked” values (True represents cells that haven’t been checked yet, False represents cells that have been checked)
- while loop:
 - pick random “True” cell; assign to array indices
 - propagate cluster from cell
 - set “checked values” array position corresponding to new cluster to false

findCluster(cell, config); do as recursive algorithm

checked algorithm idea w/Scarlett; paper (<https://journals.aps.org/prl/pdf/10.1103/PhysRevLett.62.361>)

reorganised code into file structure:

- Main
- cluster
- testing

This was done to allow for parallel development of code

things that need doing:

- cluster algorithm implementations
- dataset object implementation
- code cleanup

Scarlett will do cluster stuff, i’ll do dataset and code cleanup stuff.

got basic dataset constructor and saving/read to file functionality working

dataset has a set of:

- equilibration steps - number of steps taken to equilibrate simulation - doesn’t save configs
- simulation steps - number of steps during actual simulation

observables will be average of values over simulation steps. implemented partition function, entropy, helmholtz free energy, as observables. implemented “average” observable methods - they get the average of a given observable between simulation step indices Dataset can get standard observables from config now

To do for next week:

- Scarlett continues clustering analysis
- I continue dataset stuff and main code cleanup
 - NEED to comment/document out code
- i read up on theory

01/02/2026 notes:

Read up on theory Scarlett reviewed during Wk1 and added relevant theory to (Wk 2.2 Additional reading). Have further read into some of the critical exponents theory as well in order to develop an understanding of how to compute them from our model. The critical exponent theory follows heavily from (Rieger and Young 1993) which seems to handle scaling phenomena. As it stands, there may be two approaches to computing the critical exponents:

- Simple case; vary models over temperature and fit some variation of $(T - T_c)^x$ where x is a critical parameter to various observables (as seen in (Christensen and Moloney 2005) P132-134)
- Attempt to follow the procedure done by (Rieger and Young 1993)

The former option is simpler and is a natural extension of current work w/dataset object; however am unsure if it'd accomodate finite scaling effects. It may be worth implementing the former case first in order to test the idea; furthermore graphing observables over temperature may give a better idea of the dynamics being explored; so even if this simpler method fails, it may yield experience and insight into the system's behaviour.

Note reading around topic briefly the "Swendsen-Wang" and "Wolff" algorithms are mentioned; they seem to be used to improve compute times near critical temperatures. Am unsure if they'd be necessary; but noting their existence/potential use case here in case they'd be useful later.

A pure computational approach to extracting critical exponents may thus follow:

- create datasets of dimension d , size L over temperature ranges - try and implement a "critical temp finding" method which performs a binary search for the critical temperature in a data range for a given observable
- fit the simple critical exponent function to the data
- perform above for a range of sizes for same dimension; compare results
- if scaling seems predictable; computational approach may be sufficient, and exploration of higher dimension may begin
- else will need to read up more on how to avoid scaling effects

Therefore to do list follows:

- dataset object: finish off; it should be able to return a pandas dataframe of observables over temperature
- may be useful to generalise dataset object so it may vary model size instead of temperature
- introduce plotting methods for observables
- introduce "critical temperature finding" methods - comparing derivatives of observables at extremal ends of data, should be able to determine if current location is above/below critical temperature - this would allow a binary search for the critical temperature to arbitrary precision

- fitting methods of observables to critical temp expressions
- further cleanup of code

Lab log 02/02/2026:

tidied up dataset object - introduced a to dataframe method; and cleaned up file saving system.

dataset object testing

Starting to generate experimental datasets to gain experience generating large datasets (naming format; :

- testData20x20_1000_100: small 20x20 dataset run over 1000 eq steps, and simulated for 100 steps; range of 20 $\beta \in [0, 3]$ - magnetisation doesn't plot neatly
- testData50x50_1000_100: small 50x50 dataset run over 1000 eq steps, and simulated for 100 steps; range of 20 $\beta \in [0, 3]$ - magnetisation doesn't plot neatly

neither test case thus far plots neatly as seen:

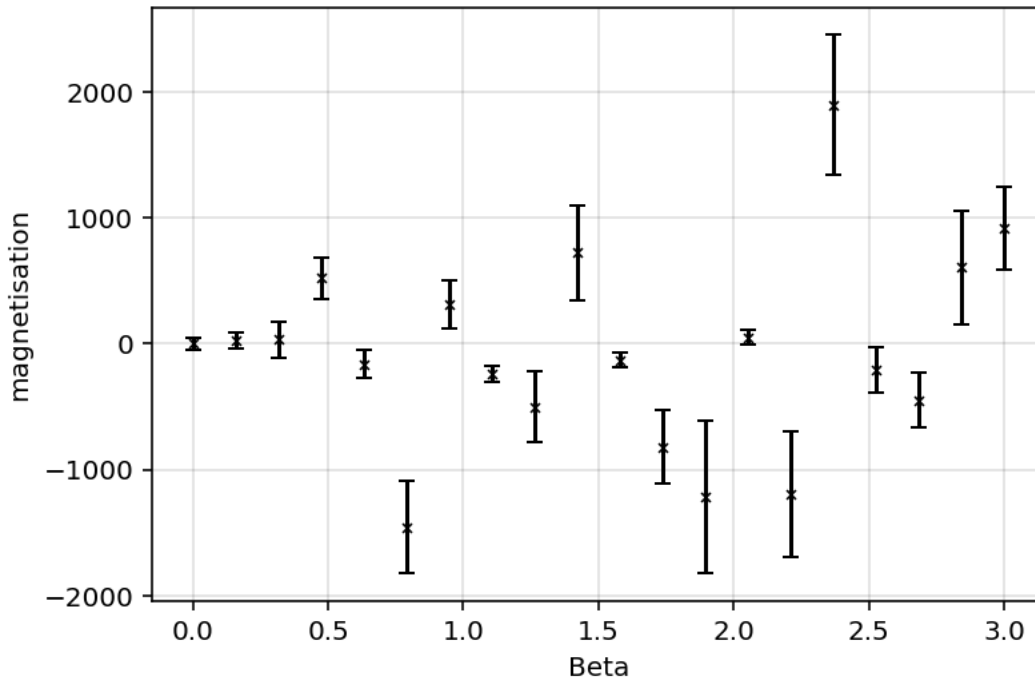


Figure 1: Fig 2.1 - Magnetisation plotted over β for testData50x50_1000_100

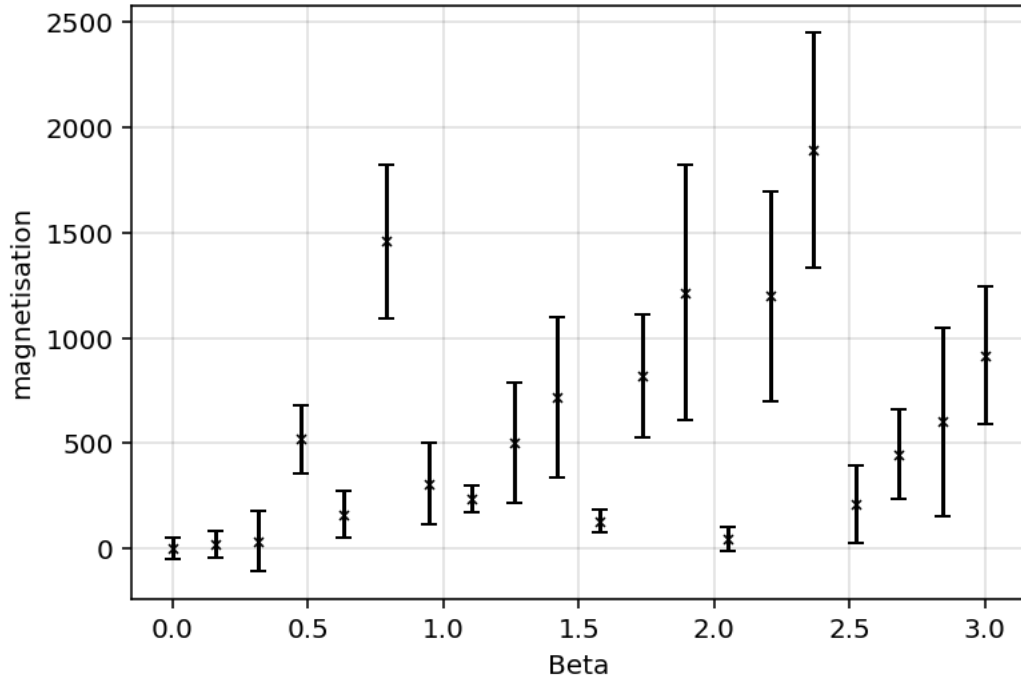


Figure 2: Fig 2.2 - Absolute Magnetisation plotted over β for testData50x50_1000_100

Which shows a general increase in absolute magnetisation over β (contrary to what intuition would suggest, with magnetisation increasing for low beta); with the occasional datapoint dropped to 0. Furthermore it is odd how absolute magnetisation increases, then suddenly drops as in fig 2.2

The final configuration states for this data was generated for visual inspection; some examples include:

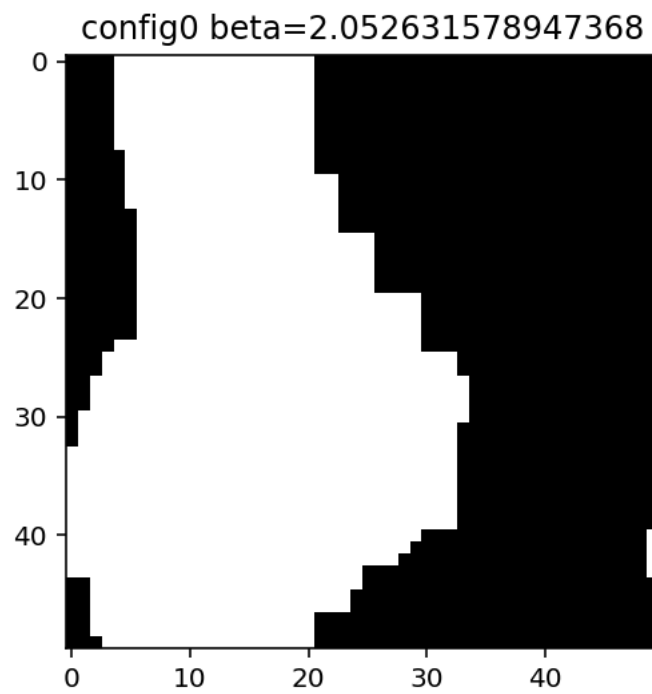


Figure 3: Fig 2.3 - Config 0

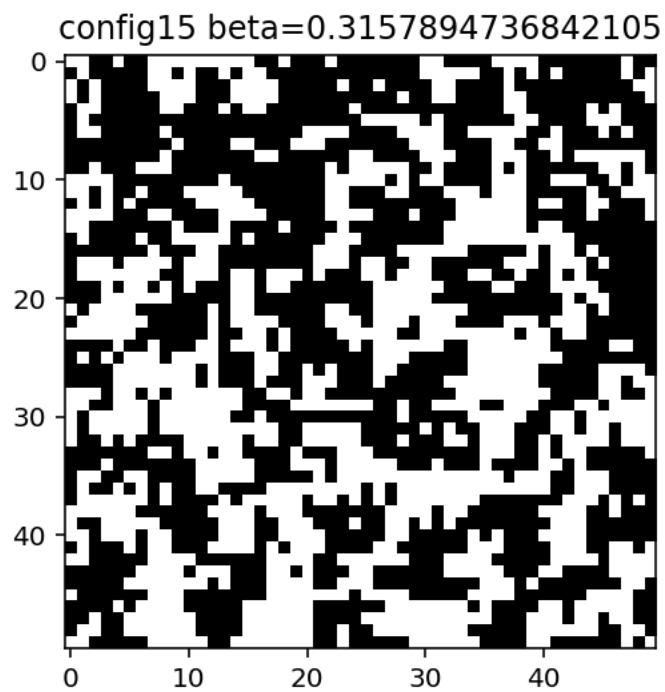


Figure 4: Fig 2.4 - Config 15

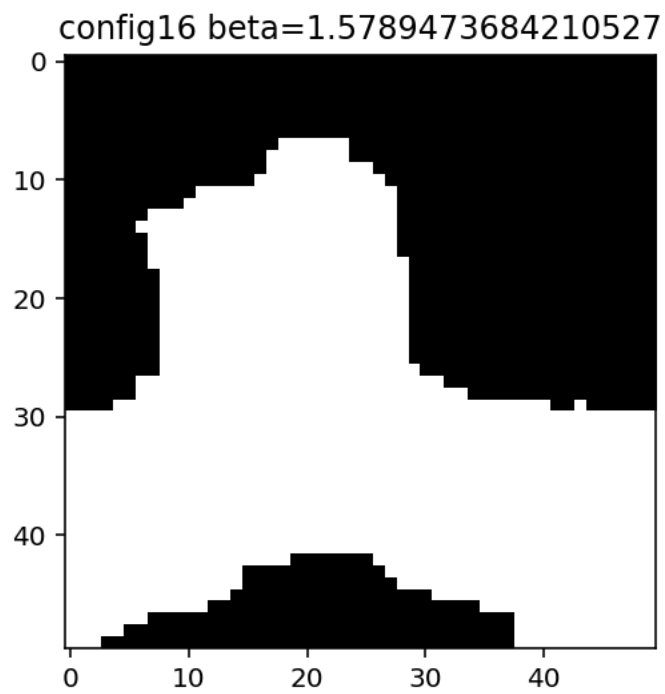


Figure 5: Fig 2.5 - Config 16

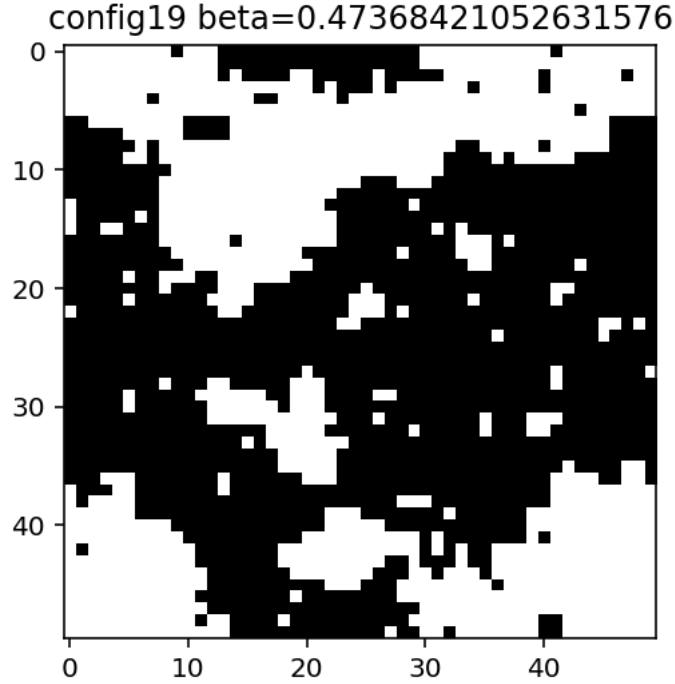


Figure 6: Fig 2.6 - Config 19

On initial inspection the low and high β configs appear reasonable; fig.2.3 shows an orderly config for low β whilst fig 2.6 shows a relatively disordered config. However the intermediate examples show a confusing switch in behaviour from relatively disorganised to organised (these correspond to one of the drops in fig 2.2).

A few things become apparent comparing figs 2.3-2.6 to fig 2.1:

- magnetisation computation is going wrong somewhere; as highly ordered states (I.E fig 2.3) are reading low magnetisation whilst disordered states (I.E fig 2.5) are reading high magnetisation. This contradicts an understanding of the underlying theory that ordered ising models should be more magnetised
- the source of the discontinuous drops are unclear - there is a major drop at $\beta \approx 2.4$ alongside several smaller drops; if those smaller drops didn't exist, that large drop could be attributed to a phase transition

The random drops may be due to noise effects (the magnetisation is averaged over the relatively few 100 simulation steps); however the fact that the magnetisation grows over beta isn't clear.

Performing larger test:

- testData50x50_1000_1000: note over range $\beta \in [0, 4]$

am hoping this will fix some of the data variability and give a clearer picture of phase transitions (even if the magnetisation over β relation is flipped)

am an absolute idiot; $\beta \propto \frac{1}{T}$; so lower β cooresponds to higher T ; so the pattern seen is as expected - nothing is broken...

additional code development ideas

May need to modify dataset object to allow different configs to have different equilibration times; (Rieger and Young 1993) suggests that equilibration time may vary over parameters

Furthermore may need to implement system to estimate equilibration times:

- dual system introduced by (Rieger and Young 1993) - create 2 configs; one starting all spin up, one all spin down; compare magnetisations of two results until they are within error of each other
- may be worth implementing as purpose built system - i suspect that the config code can be streamlined purely for the purpose of computing these equilibration times
- may also be worth creating a “equilibrationTime dataset” - able to generate a set of dualConfigs identical to dataset parameters; and spit out equilibration times - maybe make it suck that it can linearly interpolate between equilibrationTime datapoints; would allow for sparser data

Christensen, Kim, and Nicholas R Moloney. 2005. *Complexity and Criticality*. Imperial College Press. <https://doi.org/10.1142/p365>.

Domb, C, and E Stoll. 1977. “Shape and Size of Clusters in the Ising Model.” *Journal of Physics A: Mathematical and General*. <https://doi.org/10.1088/0305-4470/10/7/012>.

Rieger, H, and A P Young. 1993. “Critical Exponents of the Three-Dimensional Random Field Ising Model.” *Journal of Physics A: Mathematical and General*. <https://doi.org/10.1088/0305-4470/26/20/014>.